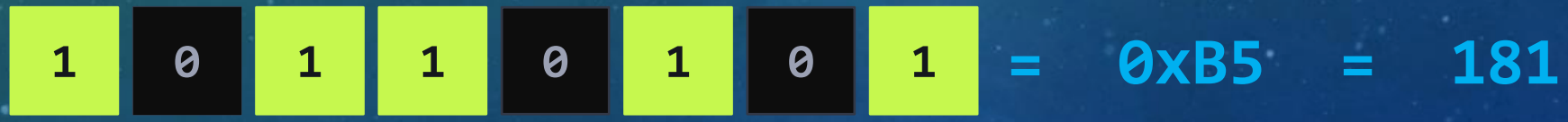



# Binary - Decimal - Hex

*A guide to how computers count and how to bend the bits.*



FOUNDATIONS

# What is a number system?

A number system represents values using a fixed set of digits. The **base** (or radix) tells you how many digits exist and how much each position is worth.

*Positional notation: each digit is multiplied by the base raised to its position.*

**Decimal 253 (base 10)**

→ $2\times10^2 + 5\times10^1 + 3\times10^0$

→ 200 + 50 + 3 = 253

*Same idea, any base: just change the 10.*

BASE 2

**Binary**

digits 0-1

BASE 10

**Decimal**

digits 0-9

BASE 16

**Hexadecimal**

digits 0-9, A-F

BASE 10

# Decimal — the everyday system

Ten digits (0–9) and the one we count with by hand. Each place is a power of **10**, growing right to left.

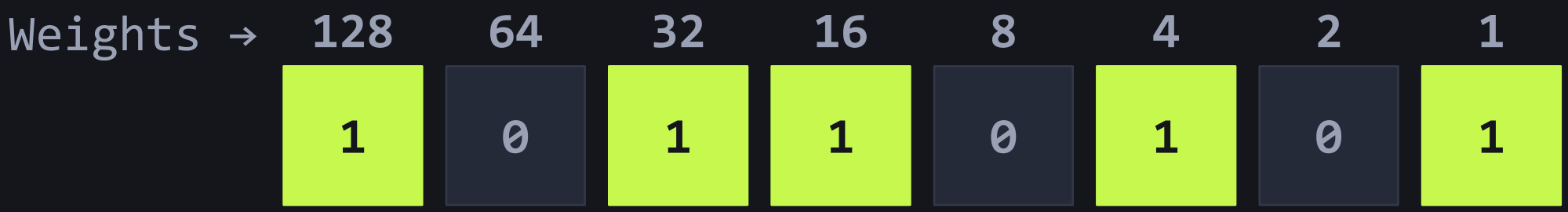
| Place | Thousands $10^3$ | Hundreds $10^2$ | Tens $10^1$ | Ones $10^0$ |
|---|---|---|---|---|
| Digit | 4 | 0 | 9 | 7 |
| Value | 4000 | 0 | 90 | 7 |

**4097** = 4000 + 0 + 90 + 7

BASE 2

# Binary — the computer's language

Only two digits: **0 and 1**. A single digit is a **bit**; eight bits make a **byte**. Each place is a power of 2.




```
128 + 32 + 16 + 4 + 1  =  181   (decimal)
```

*Add up the weights sitting above every 1. Under the hood these are just transistors switching on and off.*

BASE 16

# Hexadecimal — binary, but compact

Sixteen digits: **0–9** then **A–F** for 10–15.
One hex digit maps to exactly four bits (a **nibble**).

```
0xFF
= 1111 1111   (binary)
= 255         (decimal)
Two hex digits = one byte:
0xB5 = 1011 0101
```

## WHERE YOU'LL SEE IT

| | |
|---|---|
| Color codes | `#FF8800` → orange |
| Memory addresses | `0x7FFE1A20` |
| Byte / data values | `0xDEADBEEF` |
| Bit masks | `0x0F`, `0xF0` |

CHEAT SHEET

# 0 to 15 at a glance

Every 4-bit pattern maps to exactly one hex digit. Memorising this table makes conversions instant.

| Dec | Binary | Hex |
|---|---|---|
| 0 | 0000 | 0 |
| 1 | 0001 | 1 |
| 2 | 0010 | 2 |
| 3 | 0011 | 3 |
| 4 | 0100 | 4 |
| 5 | 0101 | 5 |
| 6 | 0110 | 6 |
| 7 | 0111 | 7 |

| Dec | Binary | Hex |
|---|---|---|
| 8 | 1000 | 8 |
| 9 | 1001 | 9 |
| 10 | 1010 | A |
| 11 | 1011 | B |
| 12 | 1100 | C |
| 13 | 1101 | D |
| 14 | 1110 | E |
| 15 | 1111 | F |

CONVERSIONS

# Converting between bases

### Decimal → Binary

Divide by 2, keep remainders, read bottom-up.

```
13 ÷ 2 = 6 r 1
6  ÷ 2 = 3 r 0
3  ÷ 2 = 1 r 1
1  ÷ 2 = 0 r 1
→ 1101
```

### Binary → Decimal

Add the weights above each 1 bit.

```
1101
8 + 4 + 0 + 1
→ 13
```

### Binary → Hex

Group bits into nibbles of 4, convert each.

```
1011 0101
1011=B  0101=5
→ 0xB5
```

BIT LOGIC 1

# Bitwise operators — AND & OR

## AND &

Result bit is 1 only when BOTH input bits are 1.

| A | B | A&B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

```
  1100
& 1010
------
= 1000
```

*Great for masking — keep only the bits you care about.*

## OR |

Result bit is 1 when EITHER input bit is 1.

| A | B | A\|B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

```
  1100
| 1010
------
= 1110
```

*Great for setting flags — turn specific bits on.*

BIT LOGIC 2

# Bitwise operators — XOR & NOT

## XOR ^

Result bit is 1 only when the input bits DIFFER.

| A | B | A^B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

```
  1100
^ 1010
------
= 0110
```

*Toggles bits; XOR-ing twice restores the original value.*

## NOT ~

Unary — flips every bit (0↔1). Works on one value.

| A | B | ~A |
|---|---|---|
| 0 | - | 1 |
| 1 | - | 0 |

```
~ 0000 1100
-----------
= 1111 0011
```

*In 8 bits, ~12 = 243. The width matters for the result.*

MOVING BITS

# Shift operators — << and >>

Shifting slides every bit left or right. Left shift fills zeros on the right; right shift drops bits off the end.

**LEFT SHIFT** `x << n` — multiplies by $2^n$

`3 << 2`




**RIGHT SHIFT** `x >> n` — divides by $2^n$ (integer)

`20 >> 2`




*Tip: for signed numbers, an arithmetic right shift copies the sign bit so negatives stay negative; a logical shift always fills 0.*

IN PRACTICE

# Putting it to work: masks & flags

### Check a bit

Is a flag set?

```
if (flags & FLAG)
```

AND with a mask, test for non-zero.

### Set a bit

Turn a flag on

```
flags |= FLAG
```

OR merges the bit in without touching others.

### Clear a bit

Turn a flag off

```
flags &= ~FLAG
```

AND with the inverted mask zeros that bit.

### Toggle a bit

Flip a flag

```
flags ^= FLAG
```

XOR switches the bit to its opposite.

### Low nibble

Keep last 4 bits

```
value & 0x0F
```

Masks out the high bits, keeps 0–9/A–F.

### Multiply / divide

Fast × or ÷ by $2^n$

```
x << 3   x >> 1
```

Shifts are cheaper than * and / for powers of 2.

# Summary

- **Same value, different clothes** — Binary, decimal and hex all describe one number — just in a different base.
- **Hex is shorthand for binary** — One hex digit = 4 bits, so bytes read cleanly as two hex characters.
- **Bitwise = logic per bit** — AND, OR, XOR and NOT combine values one bit at a time.
- **Shifts move and scale** — << and >> slide bits and act like fast × or ÷ by powers of two.
- **Masks & flags are the payoff** — Set, clear, toggle and test individual bits with a single operation.